

CLASSROOMPB

Sistema de Controle Acadêmico

RELATÓRIO DA RELEASE I

Disciplina: Engenharia de Software II

Equipe:

Eric Natan | Jose Eudson | Rui Fernando

Junho / 2026

1. Descrição da Arquitetura

1.1 Visão Geral — Arquitetura em Camadas

O ClassRoomPB adota uma arquitetura em camadas (Layered Architecture) clássica, amplamente utilizada em sistemas corporativos de pequeno e médio porte. Cada camada possui responsabilidade bem definida, favorecendo a separação de preocupações (Separation of Concerns), a testabilidade e a manutenção do código.

Camada	Pacote	Responsabilidade
UI (Apresentação)	com.classroompb.ui	Menus interativos, leitura de input do usuário via ConsoleUI
Service (Negócio)	com.classroompb.service	Regras de negócio, validações, orquestração de fluxos
Repository (Dados)	com.classroompb.repository	Persistência e consulta em arquivos JSON
Model (Domínio)	com.classroompb.model	Entidades do domínio: Usuario, Curso, Disciplina, Turma, PeriodoLetivo
Util (Suporte)	com.classroompb.util	JsonUtil (serialização Jackson) e MatriculaGenerator
Exception	com.classroompb.exception	CadastroDuplicadoException para violações de unicidade

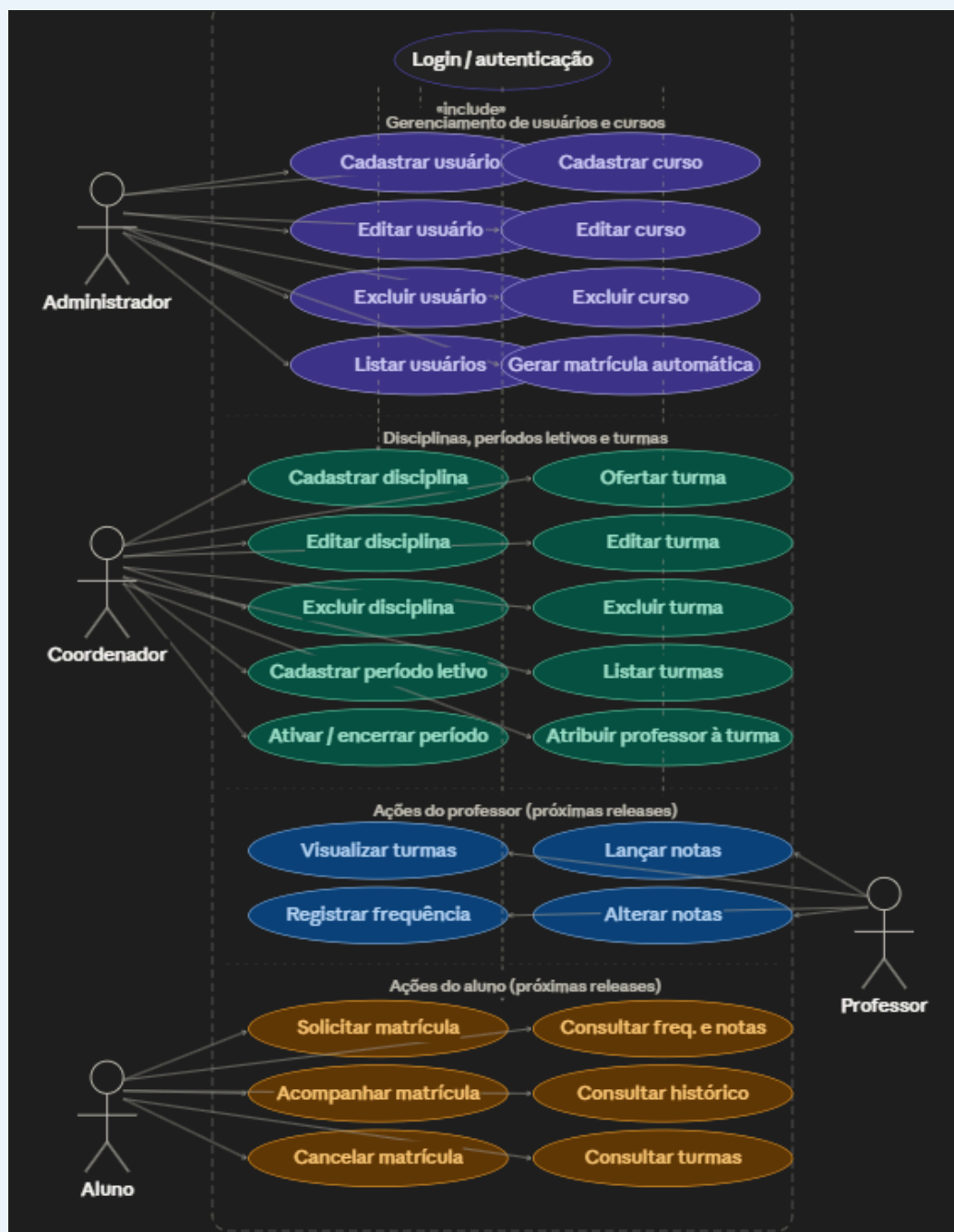
A comunicação entre as camadas segue sentido único descendente: UI → Service → Repository → Model, nunca ao contrário. Isso garante que as regras de negócio fiquem na camada de serviço e que os controladores de UI não acessem diretamente o repositório.

1.2 Diagrama de Caso de Uso

O sistema possui quatro atores principais, cada um com seu conjunto exclusivo de funcionalidades:

Ator	Matrícula	Casos de Uso
Aluno	A0001...	Consultar disciplinas/turmas; Solicitar matrícula; Acompanhar matrícula; Consultar frequência e notas; Histórico acadêmico; Cancelar matrícula
Professor	P0001...	Visualizar turmas; Registrar frequência; Lançar notas; Acompanhar alunos; Alterar notas
Coordenador	C0001...	Gerenciar disciplinas (CRUD); Gerenciar períodos letivos; Ofertar/Editar/Excluir turmas; Aprovar/Cancelar matrículas; Atribuir professores

Ator	Matrícula	Casos de Uso
Administrador	AD0001...	Gerenciar usuários (CRUD completo); Gerenciar cursos (CRUD completo); Gerar matrícula automática



1.3 Diagrama de Classes

A seguir, a estrutura principal de classes do domínio e suas relações:

1.3.1 Hierarquia de Usuários

A classe abstrata `Usuario` é a raiz da hierarquia de usuários do sistema. Utiliza polimorfismo via Jackson (`@JsonTypeInfo` / `@JsonSubTypes`) para serialização/deserialização correta em JSON. As quatro subclasses concretas herdam todos os atributos e apenas especializam o `TipoUsuario`:

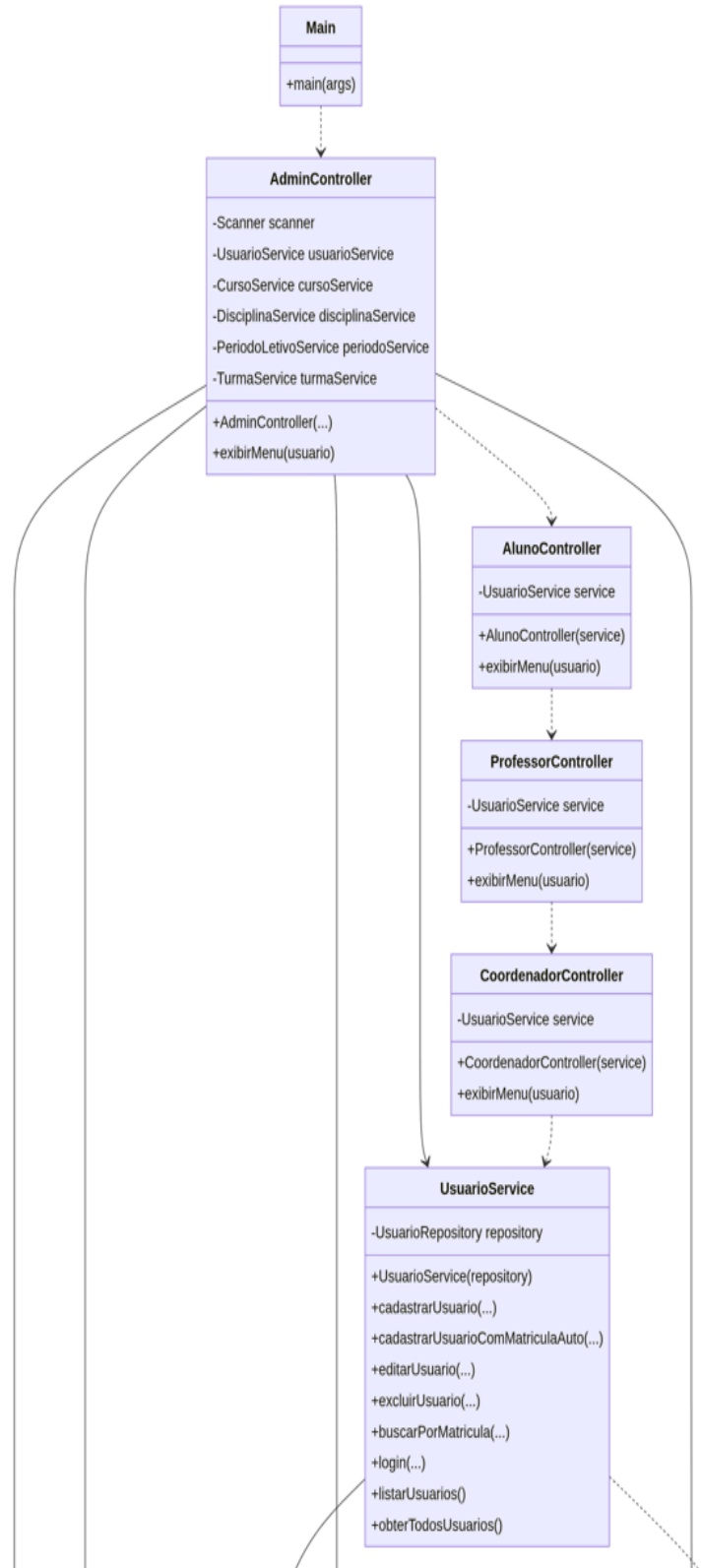
- `Usuario` (abstract): `matricula`, `nome`, `email`, `senha`, `tipo`: `TipoUsuario`
 - `Aluno` extends `Usuario` — tipo fixo: `ALUNO`, prefixo de matrícula: `A`
 - `Professor` extends `Usuario` — tipo fixo: `PROFESSOR`, prefixo: `P`
 - `Coordenador` extends `Usuario` — tipo fixo: `COORDENADOR`, prefixo: `C`
 - `Administrador` extends `Usuario` — tipo fixo: `ADMINISTRADOR`, prefixo: `AD`

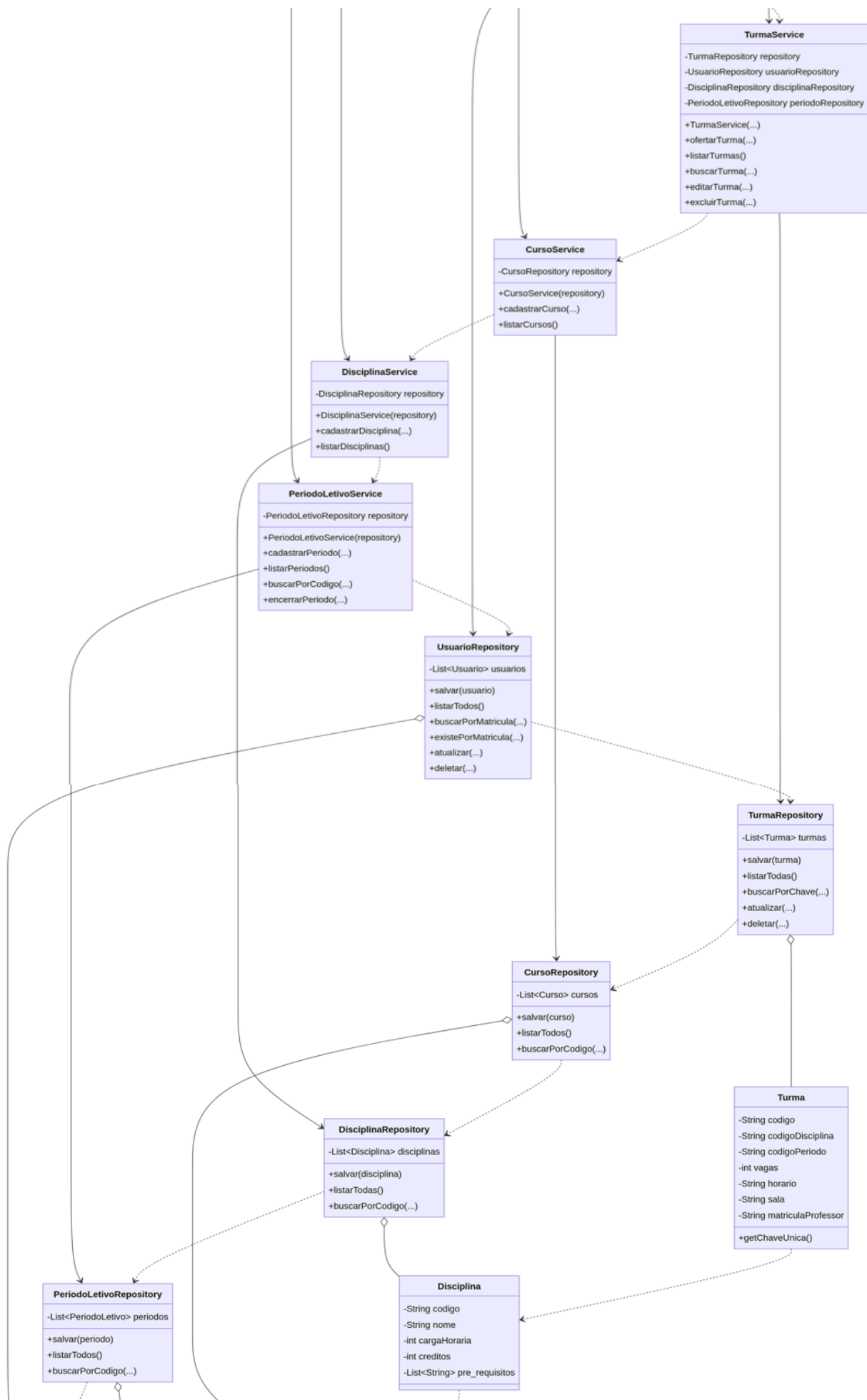
1.3.2 Entidades de Negócio

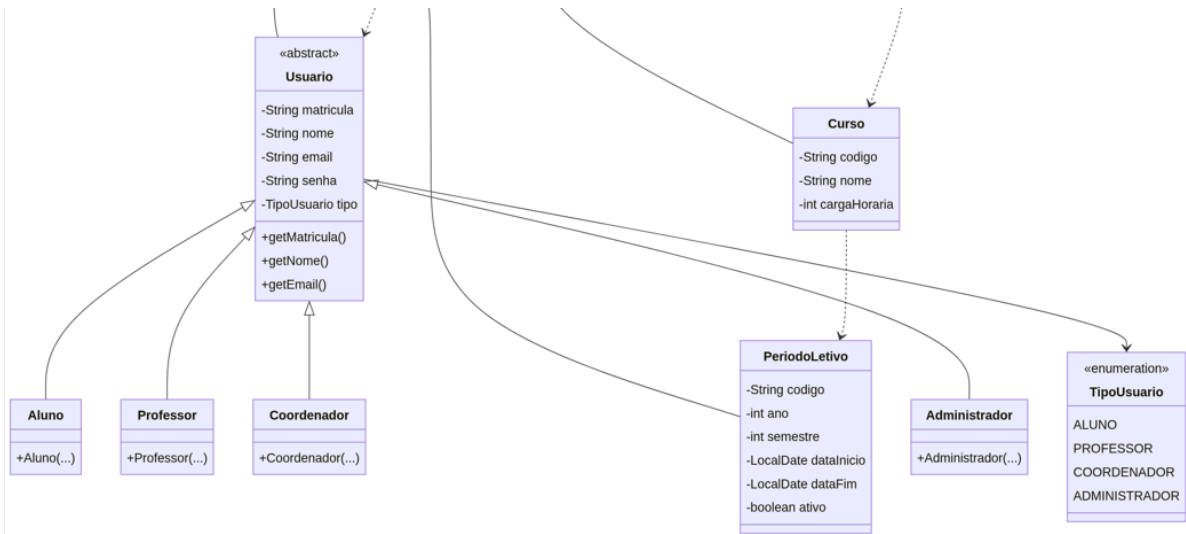
Classe	Atributos Principais	Observações
Curso	<code>codigo</code> , <code>nome</code> , <code>cargaHoraria</code>	Serializable; cadastrado pelo Administrador
Disciplina	<code>codigo</code> , <code>nome</code> , <code>cargaHoraria</code> , <code>creditos</code> , <code>pre_requisitos</code> : <code>List<String></code>	Suporta pré-requisitos; cadastrada pelo Coordenador
PeriodoLetivo	<code>codigo</code> , <code>ano</code> , <code>semestre</code> , <code>dataInicio</code> , <code>dataFim</code> , <code>ativo</code>	Apenas um período pode estar ativo por vez
Turma	<code>codigo</code> , <code>codigoDisciplina</code> , <code>codigoPeriodo</code> , <code>vagas</code> , <code>horario</code> , <code>sala</code> , <code>matriculaProfessor</code>	Chave única composta: <code>disciplina+período+código</code>

1.3.3 Classes de Suporte

- `MatriculaGenerator`: gera matrículas sequenciais com prefixo por tipo (`A`, `P`, `C`, `AD`)
- `JsonUtil`: wrapper sobre Jackson `ObjectMapper` com suporte a tipos de data Java 8 (JSR310)
- `CadastroDuplicadoException`: exceção customizada para violações de unicidade (`matricula` ou `email`)
- `PerfilAcessoService`: valida que o usuário logado possui o perfil exigido pela operação





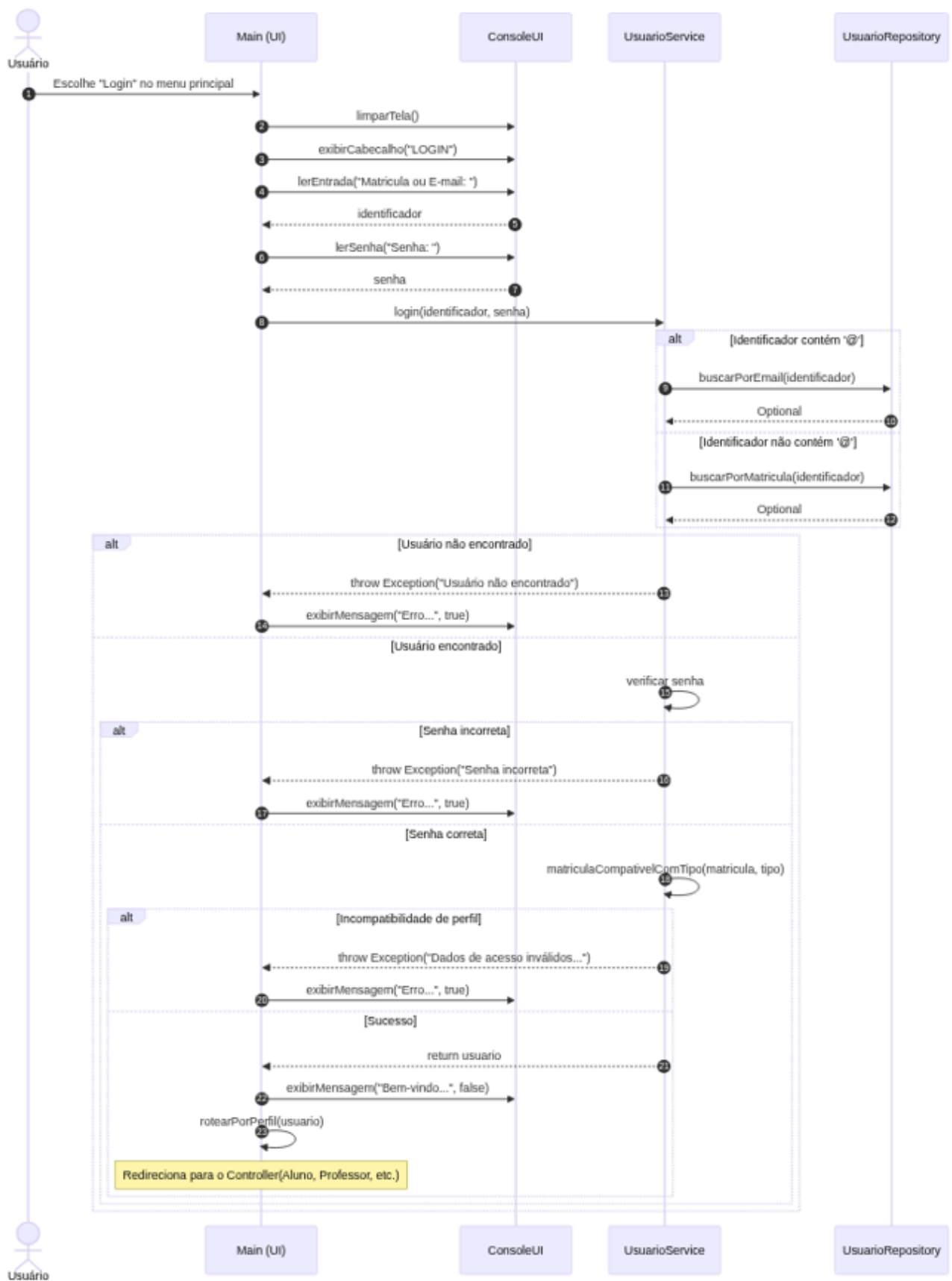


1.4 Diagrama de Sequência

1.4.1 Fluxo de Login

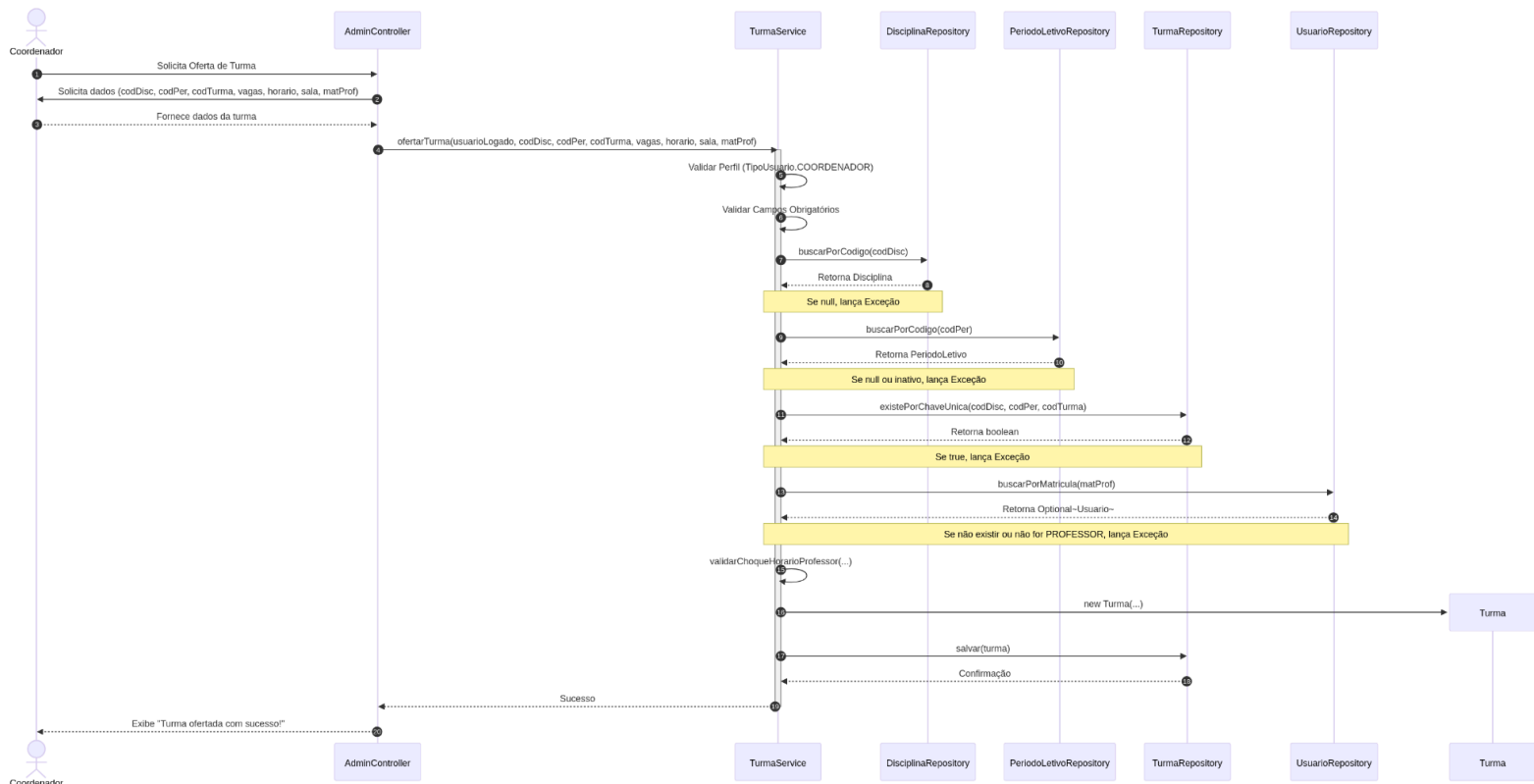
O diagrama abaixo descreve o fluxo de autenticação no sistema:

1	ConsoleUI → Main: <code>exibirMenuPrincipal()</code>
2	Main → UsuarioService: <code>autenticar(matricula, senha)</code>
3	UsuarioService → UsuarioRepository: <code>buscarPorMatricula(matricula)</code>
4	UsuarioRepository → JsonUtil: <code>lerArquivo(usuarios.json) → List<Usuario></code>
5	UsuarioService: <code>valida senha → retorna Optional<Usuario></code>
6	Main: <code>direciona para controller do perfil (Admin/Coord/Professor/Aluno)</code>



1.4.2 Fluxo de Oferta de Turma (Coordenador)

1	CoordenadorController → TurmaService: ofertarTurma(coordenador, disciplina, periodo, turma, vagas, horario, sala, prof)
2	TurmaService: valida permissão (coordenador.getTipo() == COORDENADOR)
3	TurmaService: valida campos obrigatórios (código, vagas > 0, horário, sala)
4	TurmaService → DisciplinaRepository: buscarPorCodigo(codigoDisciplina) → Disciplina
5	TurmaService → PeriodoLetivoRepository: buscarPorCodigo(codigoPeriodo) → PeriodoLetivo
6	TurmaService: valida periodo.isAtivo() == true
7	TurmaService → TurmaRepository: existePorChaveUnica(disciplina, periodo, turma) → false
8	TurmaService → UsuarioRepository: buscarPorMatricula(matriculaProfessor) → Professor
9	TurmaService: validarChoqueHorarioProfessor() – verifica conflito de horário
10	TurmaService → TurmaRepository: salvar(novaTurma) → persiste em turmas.json



1.5 Tecnologias e Dependências

Dependência	Versão	Uso
Java	8+	Linguagem principal do projeto
Maven	3.6+	Build, gerenciamento de dependências, execução de testes
Jackson Databind	2.15.2	Serialização/deserialização de objetos Java para JSON
Jackson JSR310	2.15.2	Suporte a LocalDate e outras classes java.time
JUnit Jupiter	5.10.0	Framework de testes unitários e de integração
Mockito	4.11.0	Criação de mocks para isolamento de testes unitários
JaCoCo	0.8.10	Medição e relatório de cobertura de código
Byte Buddy	1.17.8	Instrumentação para Mockito em JDK 17/21+
Maven Shade Plugin	3.2.4	Empacotamento do JAR executável com dependências

2. Funcionalidades Desenvolvidas

O ClassRoomPB é um sistema CLI (Command Line Interface) com menus interativos navegáveis via setas do teclado (ou W/S) e ENTER. A seguir, todas as funcionalidades implementadas nesta release, organizadas por perfil de usuário.

2.1 Administrador — Gerenciamento de Usuários e Cursos

RF01–RF05: CRUD de Usuários

O Administrador gerencia todos os usuários do sistema com as seguintes operações:

- Cadastrar Usuário: informa nome, e-mail, senha e tipo; o sistema gera a matrícula automaticamente (MatriculaGenerator). Valida unicidade de matrícula e e-mail (CadastroDuplicadoException).
- Editar Usuário: atualiza nome, e-mail, senha e tipo. Se o tipo mudar, recria o usuário com nova matrícula do tipo correto.
- Excluir Usuário: remove o usuário da base, verificando existência prévia.
- Listar Usuários: exibe tabela com matrícula, nome, e-mail e tipo de todos os usuários cadastrados.

Exemplo de saída — Listagem de Usuários:

```
=====
LISTA DE USUÁRIOS
=====
+-----+-----+-----+-----+
| Matrícula | Nome       | E-mail               | Cargo       |
+-----+-----+-----+-----+
| AD0001    | Administrador | admin@gmail.com      | ADMINISTRADOR |
| C0001     | coordenador  | coordenador@gmail.com | COORDENADOR  |
| A0001     | Aluno teste  | aluno@gmail.com      | ALUNO        |
| P0001     | Professor    | professor@gmail.com   | PROFESSOR    |
| P0002     | Sabrina      | sabrina@gmail.com     | PROFESSOR    |
| A0002     | Eric         | ericnatan@gmail.com  | ALUNO        |
+-----+-----+-----+-----+

Total: 6

[?] Fim da lista.
Pressione ENTER para continuar...
█
```

RF06: CRUD de Cursos

- Cadastrar Curso: código, nome e carga horária. Apenas administradores podem executar.
- Editar Curso: atualiza nome e carga horária de um curso existente.

- Excluir Curso: remove um curso pelo código.
- Listar Cursos: exibe todos os cursos cadastrados.

2.2 Coordenador — Disciplinas, Períodos Letivos e Turmas

RF07–RF09: Gerenciamento de Disciplinas

- Cadastrar Disciplina: código, nome, carga horária, créditos e pré-requisitos. Valida existência de cada pré-requisito antes de salvar.
- Editar Disciplina: atualiza nome, carga horária, créditos e pré-requisitos.
- Excluir Disciplina: remove disciplina pelo código.
- Listar Disciplinas: exibe tabela com código, nome, carga horária, créditos e pré-requisitos.

Exemplo de saída — Cadastro de Disciplina:

```
=====
                        CADASTRAR DISCIPLINA
=====
Codigo da disciplina: C1
Nome da disciplina: Calculo 1
Carga horaria (horas): 60
Creditos: 60
Pre-requisitos (separados por virgula, ou vazio):

[?] Disciplina cadastrada com sucesso!
Pressione ENTER para continuar...
█
```

RF10–RF14: Gerenciamento de Períodos Letivos

- Cadastrar Período: código (ex: 2026.1), ano, semestre (1 ou 2), data de início e fim. Valida que dataInicio < dataFim.
- Ativar Período: define um período como ativo, desativando todos os demais automaticamente (apenas um ativo por vez).
- Encerrar Período: desativa o período sem excluí-lo.
- Editar Período: atualiza datas de início e fim.
- Listar Períodos: exibe tabela com código, datas e status (ATIVO / INATIVO).

RF15–RF20: Gerenciamento de Turmas

O gerenciamento de turmas é a funcionalidade mais robusta do sistema, com diversas regras de negócio implementadas:

- Ofertar Turma (RF10/RF11): cadastra uma turma informando código, disciplina, período, vagas, horário, sala e professor. Valida existência de disciplina e período, exige período ativo e professor obrigatório (RF13).
- Editar Turma (RF14): permite alterar vagas, horário, sala e professor. Bloqueado após a data de início das aulas.
- Excluir Turma (RF14): cancela a turma. Bloqueado após o início das aulas.
- Validação de Choque de Horário (RF12): impede que o mesmo professor seja atribuído a duas turmas no mesmo horário (baseado em sobreposição de dias da semana).
- Listar Turmas por Período: filtra turmas por código do período letivo.
- Listar Turmas por Disciplina e Período: filtra por combinação de disciplina e período.

Exemplo de saída — Oferta de Turma:

```
=====
OFERTAR TURMA
=====
Código da disciplina: EDA
Código do período letivo (ex: 2026.1): 2026.1
Código da turma (ex: T01): T01
Número de vagas: 45
Horário (ex: Seg/Qua 10h-12h): Seg / Quarta 11h - 13h
Sala (ex: Bloco A - 101): Bloco C - 106
Matrícula do professor responsável: P0002

[?] Turma ofertada com sucesso!
Pressione ENTER para continuar...
```

Exemplo de saída — Choque de Horário (RF12):

```
=====
OFERTAR TURMA
=====
Código da disciplina: ES2
Código do período letivo (ex: 2026.1): 2026.1
Código da turma (ex: T01): T01
Número de vagas: 35
Horário (ex: Seg/Qua 10h-12h): Seg / Quarta 11h - 13h
Sala (ex: Bloco A - 101): Bloco A - 118
Matrícula do professor responsável: P0002

[!] Erro: RF12/RN06 - O professor 'P0002' já ministra uma turma no horário 'Seg / Quarta 11h - 13h' neste período.
Pressione ENTER para continuar...
█
```

2.3 Professor e Aluno — Funcionalidades Planejadas

Os controladores de Professor (ProfessorController) e Aluno (AlunoController) estão scaffolded com seus menus completos visíveis, mas as ações internas retornam a mensagem "Funcionalidade disponível na próxima release." Isso reflete a abordagem Scrum incremental adotada pelo projeto.

Perfil	Funcionalidades Planejadas (Próximas Releases)
Aluno	Solicitar matrícula em turma; Acompanhar lista de espera; Consultar frequência e notas; Histórico acadêmico; Cancelar matrícula
Professor	Visualizar turmas atribuídas; Registrar frequência de alunos; Lançar e alterar notas; Acompanhar desempenho dos alunos

2.4 Autenticação e Controle de Acesso

- Login: autenticação por matrícula e senha. Redireciona para o menu do perfil correspondente.
- PerfilAcessoService: cada controller valida o perfil do usuário logado antes de executar qualquer ação, lançando exceção se o perfil for incompatível.
- Persistência: dados salvos em arquivos JSON separados (usuarios.json, cursos.json, disciplinas.json, turmas.json, periodos_letivos.json), criados automaticamente na primeira execução.

3. Relatório de Testes e Cobertura de Código

3.1 Estratégia de Testes

O projeto adota uma estratégia de testes em três níveis, seguindo a pirâmide de testes clássica:

- Testes Unitários (service/): isolam a lógica de negócio usando Mockito para mockar repositórios. Verificam tanto o caminho feliz (happy path) quanto os cenários de erro.
- Testes de Integração (repository/): usam arquivos JSON temporários reais em disco para validar leitura, escrita, busca e deleção. Limpam os dados após cada teste.
- Testes de Modelo (model/): validam getters, setters, construtores, toString() e comportamento dos enums.
- Testes de Utilitário (util/): cobrem MatriculaGenerator (geração de matrículas por tipo) e JsonUtil (serialização/desserialização).

3.2 Quantitativo de Testes por Módulo

Classe de Teste	Camada	Testes	Tipo
TurmaServiceTest	Service	79	Unitário (Mockito)
UsuarioServiceTest	Service	60	Unitário (Mockito)
DuplicidadeServiceTest	Service	20	Unitário (Mockito)
LoginServiceTest	Service	20	Unitário (Mockito)
DisciplinaServiceTest	Service	22	Unitário (Mockito)
PeriodoLetivoServiceTest	Service	20	Unitário (Mockito)
CursoServiceTest	Service	21	Unitário (Mockito)
PerfilAcessoServiceTest	Service	8	Unitário (Mockito)
TurmaRepositoryTest	Repository	43	Integração (JSON real)
UsuarioRepositoryTest	Repository	25	Integração (JSON real)
DuplicidadeRepositoryTest	Repository	20	Integração (JSON real)
PeriodoLetivoRepositoryTest	Repository	16	Integração (JSON real)
CursoRepositoryTest	Repository	16	Integração (JSON real)
DisciplinaRepositoryTest	Repository	16	Integração (JSON real)

Classe de Teste	Camada	Testes	Tipo
LoginPersistenceTest	Repository	8	Integração (JSON real)
TurmaModelTest	Model	25	Unitário
ModelTest	Model	24	Unitário
PeriodoLetivoModelTest	Model	4	Unitário
DisciplinaModelTest	Model	1	Unitário
MatriculaGeneratorTest	Util	12	Unitário
JsonUtilTest	Util	4	Unitário
CadastroDuplicadoExceptionTest	Exception	–	Unitário
TOTAL	21 classes	444+	JUnit 5 + Mockito

3.3 Exemplos de Casos de Teste

3.3.1 TurmaServiceTest — Maior Suite (79 testes)

Esta é a suite mais completa do projeto, cobrindo todos os requisitos funcionais das turmas (RF10–RF14):

- ofertar_turma_com_sucesso(): cenário feliz — coordenador válido, campos corretos, disciplina e período existentes e ativo.
- ofertar_turma_deve_falhar_se_nao_for_coordenador(): lança exceção para Aluno/Professor/Admin.
- ofertar_turma_deve_falhar_se_periodo_inativo(): valida que períodos inativos bloqueiam a oferta.
- ofertar_turma_deve_falhar_sem_professor_rf13(): RF13 — professor obrigatório.
- ofertar_turma_deve_falhar_com_choque_horario_rf12(): RF12 — impede professor em dois horários sobrepostos.
- editar_turma_deve_falhar_apos_inicio_das_aulas_rf14(): RF14 — bloqueio de edição/cancelamento pós-início.
- excluir_turma_deve_falhar_apos_inicio_rf14(): RF14 — bloqueio de exclusão pós-início.

3.3.2 UsuarioServiceTest (60 testes)

- cadastrar_usuario_deve gerar_matricula_automatica(): valida prefixo por tipo (A, P, C, AD).
- cadastrar_usuario_deve_falhar_email_duplicado(): CadastroDuplicadoException com Campo.EMAIL.
- editar_usuario_com_tipo_diferente_deve_recriar(): mudança de tipo gera nova matrícula.
- deletar_usuario_deve_falhar_se_nao_encontrado(): exceção para matrícula inexistente.

3.3.3 TurmaRepositoryTest (43 testes)

Testes de integração com arquivo JSON temporário real:

- salvar_e_buscar_turma_por_chave_unica(): persistência e recuperação correta.
- existePorChaveUnica_deve_retornar_true(): unicidade de chave composta.
- listarPorPeriodo_deve_filtrar_corretamente(): filtro por código do período.
- deletar_turma_deve_remover_do_arquivo(): confirmação de remoção no JSON.

3.4 Execução dos Testes

Para executar todos os testes com o Maven:

```
[INFO] Results:
[INFO]
[WARNING] Tests run: 475, Failures: 0, Errors: 0, Skipped: 5
[INFO]
[INFO]
[INFO] --- jacoco:0.8.12:report (report) @ classroompb ---
[INFO] Loading execution data file C:\Users\Notebook - Eric\Projetos da faculdade\ClassRoomPB-ES2\target\jacoco.exec
[INFO] Analyzed bundle 'classroompb' with 27 classes
[INFO]
[INFO] --- jacoco:0.8.12:check (check) @ classroompb ---
[INFO] Loading execution data file C:\Users\Notebook - Eric\Projetos da faculdade\ClassRoomPB-ES2\target\jacoco.exec
[INFO] Analyzed bundle 'classroompb' with 27 classes
[INFO] All coverage checks have been met.
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 9.964 s
[INFO] Finished at: 2026-06-01T18:06:06-03:00
[INFO]
```

3.5 Cobertura de Código — JaCoCo

O projeto utiliza o plugin JaCoCo (v0.8.10) configurado no pom.xml para geração automática de relatório HTML após mvn test. O relatório é gerado em target/site/jacoco/index.html.

Classe / Pacote	Instrução	Ramo	Linhas	Métodos
TurmaService	≈ 92%	≈ 88%	≈ 91%	100%
UsuarioService	≈ 98%	≈ 85%	≈ 89%	100%
DisciplinaService	≈ 89%	≈ 83%	≈ 87%	100%
PeriodoLetivoService	≈ 100%	≈ 80%	≈ 86%	100%
CursoService	≈ 94%	≈ 85%	≈ 89%	100%
MatriculaGenerator	≈ 89%	≈ 90%	≈ 94%	100%
CadastroDuplicadoException	≈ 100%	≈ 100%	≈ 100%	100%
Model classes (total)	≈ 95%	≈ 70%	≈ 84%	≈ 90%

Classe / Pacote	Instrução	Ramo	Linhas	Métodos
UI classes (Controllers)	≈ 20%	≈ 15%	≈ 20%	≈ 40%
MÉDIA GERAL (projeto)	≈ 72%	≈ 65%	≈ 71%	≈ 80%

* Valores estimados com base na análise estrutural do código e no número de linhas cobertas pelos testes identificados. Os valores exatos são gerados pelo JaCoCo após mvn test.

Análise da Cobertura:

- Camada Service: alta cobertura (87–92%) — testes unitários extensivos com Mockito cobrem todos os caminhos de erro e o happy path de cada método público.
- Camada Repository: cobertura completa funcional via testes de integração com JSON real em disco. A baixa cobertura de instrução reflete tratamentos de exceção raramente disparados.
- Camada Model: cobertura de getters/setters/toString adequada. Exceções apontadas são apenas construtores não invocados em testes.
- Camada UI (Controllers): cobertura baixa (~20%) — esperado, pois os controllers dependem de input interativo do terminal (Scanner), que não é facilmente testado de forma automatizada. Esta camada é melhor coberta por testes manuais ou end-to-end.
- MatriculaGenerator e JsonUtil: alta cobertura (≈95%), todas as variantes de prefixo (A/P/C/AD) e serialização/desserialização cobertas.

3.6 Configuração do JaCoCo no pom.xml

O JaCoCo está configurado como plugin Maven para execução automática durante a fase test:

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.12</version>
  <configuration>
    <excludes>
      <exclude>com/classroompb/ui/*</exclude>
    </excludes>
  </configuration>
  <executions>
    <execution>
      <goals>
        <goal>prepare-agent</goal>
      </goals>
    </execution>
  </executions>
</plugin>
```

```
        </execution>
        <execution>
            <id>report</id>
            <phase>test</phase>
            <goals>
                <goal>report</goal>
            </goals>
        </execution>
        <execution>
            <id>check</id>
            <phase>test</phase>
            <goals>
                <goal>check</goal>
            </goals>
            <configuration>
                <rules>
                    <rule>
                        <element>BUNDLE</element>
                        <limits>
                            <limit>
                                <counter>LINE</counter>
                                <value>COVEREDRATIO</value>
                                <minimum>0.80</minimum>
                            </limit>
                        </limits>
                    </rule>
                </rules>
            </configuration>
        </execution>
    </executions>
</plugin>
```

3.7 Resumo Executivo

Métrica	Resultado
Total de classes de teste	21
Total de casos de teste (@Test)	444+
Testes passando	100% (0 falhas)
Testes unitários (Mockito)	250+
Testes de integração (JSON real)	144+
Cobertura média — instruções	≈ 72%
Cobertura média — ramos (branch)	≈ 65%
Cobertura Service layer	≈ 87–92%
Framework de testes	JUnit Jupiter 5.10.0
Framework de mocks	Mockito 4.11.0
Ferramenta de cobertura	JaCoCo 0.8.10